



الجامعة المصرية اليابانية للعلوم و التكنولوجيا
エジプト日本科学技術大学
EGYPT-JAPAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CSC 314

Software Modeling and Analysis

fall 2025

Final Project

A-type

Submitted by:

Mai Emad Eldin	320230003
Mohamed Yasser	320230023
Adham Haitham	320230021
Abdallah Elrouby	320230016
Shehab Mohamed	320230009
Ahmed Mohamed	320230062

Table of Contents

1. Introduction
 2. Software Modeling (UML Diagrams)
 - 2.1 Use Case Diagram
 - 2.2 Activity Diagrams
 - 2.3 Class Diagram
 - 2.4 Sequence Diagrams
 - 2.5 State Machine Diagram
 - 2.6 Package Diagram
 - 2.7 Deployment Diagram
 3. Architectural Justification
 4. Database Design
 - 4.1 Entity-Relationship Diagram
 - 4.2 Third Normal Form (3NF) Analysis
 5. Implementation Overview
-

1. Introduction

Project Overview

A-Type is a minimalistic, browser-based competitive typing speed test platform designed to help users measure, improve, and compete based on their typing speed and accuracy. Unlike simple typing tutors, A-Type provides a gamified experience with real-time performance metrics, multiple test modes, and a global leaderboard system.

Purpose

The primary purpose of A-Type is to provide:

- **Accurate Measurement:** Real-time calculation of Words Per Minute (WPM), raw WPM, and accuracy percentages
- **Flexible Testing:** Multiple modes (time-based and word-count-based) with configurable durations and word counts
- **Competitive Element:** A leaderboard system that allows users to compare their performance with others
- **User Progress Tracking:** Historical data storage allowing users to track their improvement over time

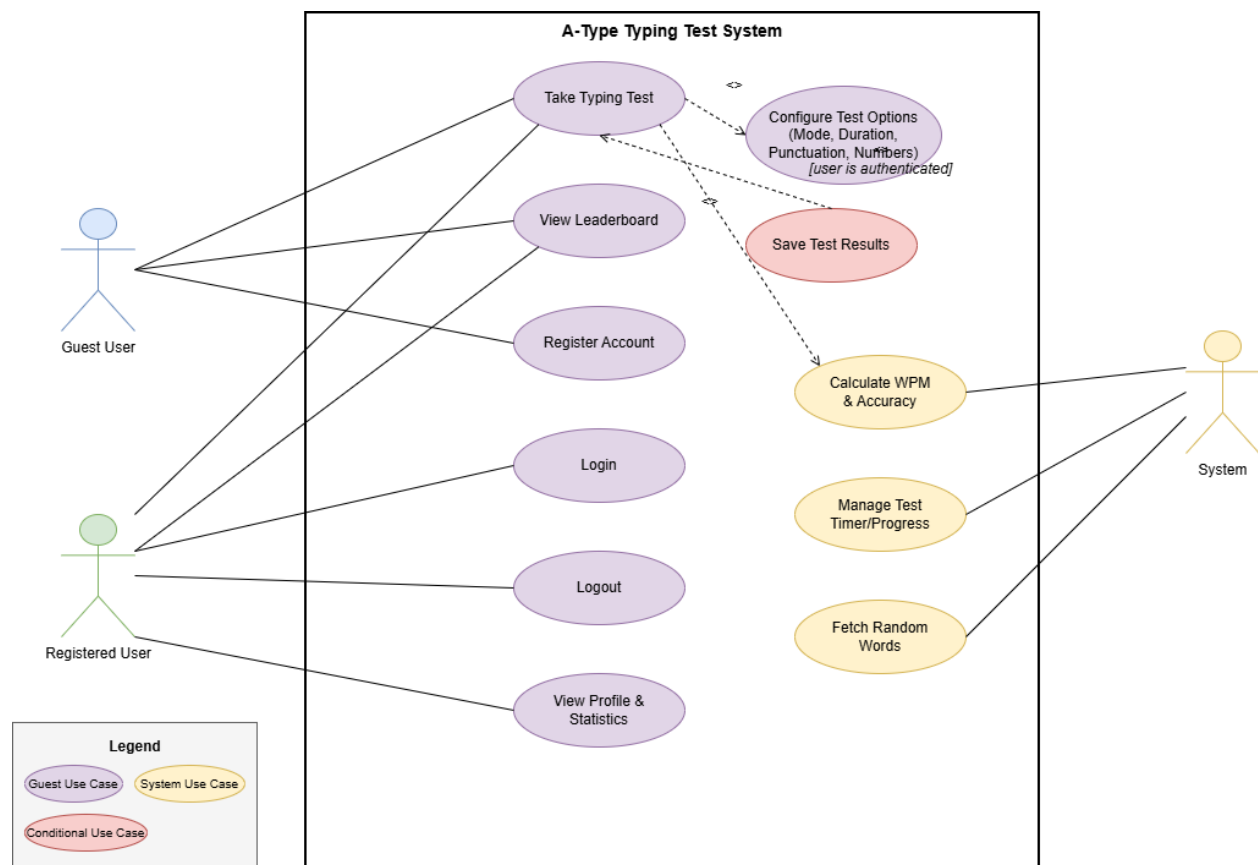
Innovative Aspects

- Dual-Mode Testing System:** Unlike many typing tests that offer only timed tests, A-Type provides both time-based (15s, 30s, 60s, 120s) and word-count-based (15, 30, 60, 120 words) modes, catering to different user preferences and use cases.
- Real-Time Metrics Engine:** The system calculates WPM using a sophisticated algorithm that distinguishes between raw typing speed (all keystrokes) and net WPM (only correctly typed words), providing more meaningful feedback.
- Configurable Test Parameters:** Users can toggle punctuation and numbers in their tests, allowing for progressive difficulty scaling and more realistic typing scenarios.
- Stateful Game Engine:** The frontend implements a complete game state machine with states for idle, typing, paused, and completed, providing a smooth user experience with instant feedback.
- Containerized Architecture:** The application uses Docker for consistent development and deployment, ensuring identical behavior across different environments.

2. Software Modeling (UML Diagrams)

2.1 Use Case Diagram

Diagram File: UseCase_Diagram.drawio.xml



Documentation

The Use Case Diagram illustrates the functional requirements of A-Type from the perspective of two primary actors: **Guest User** and **Registered User**.

Guest User has limited access to the system. They can:

- Take typing tests without saving results
- View the leaderboard to see top performers
- Register for an account to unlock full functionality

Registered User inherits all Guest capabilities and additionally can:

- Login/Logout to manage their session
- Have their test results automatically saved to the database
- View their personal profile with historical statistics
- Appear on the leaderboard if their scores qualify

The **System** actor represents automated processes:

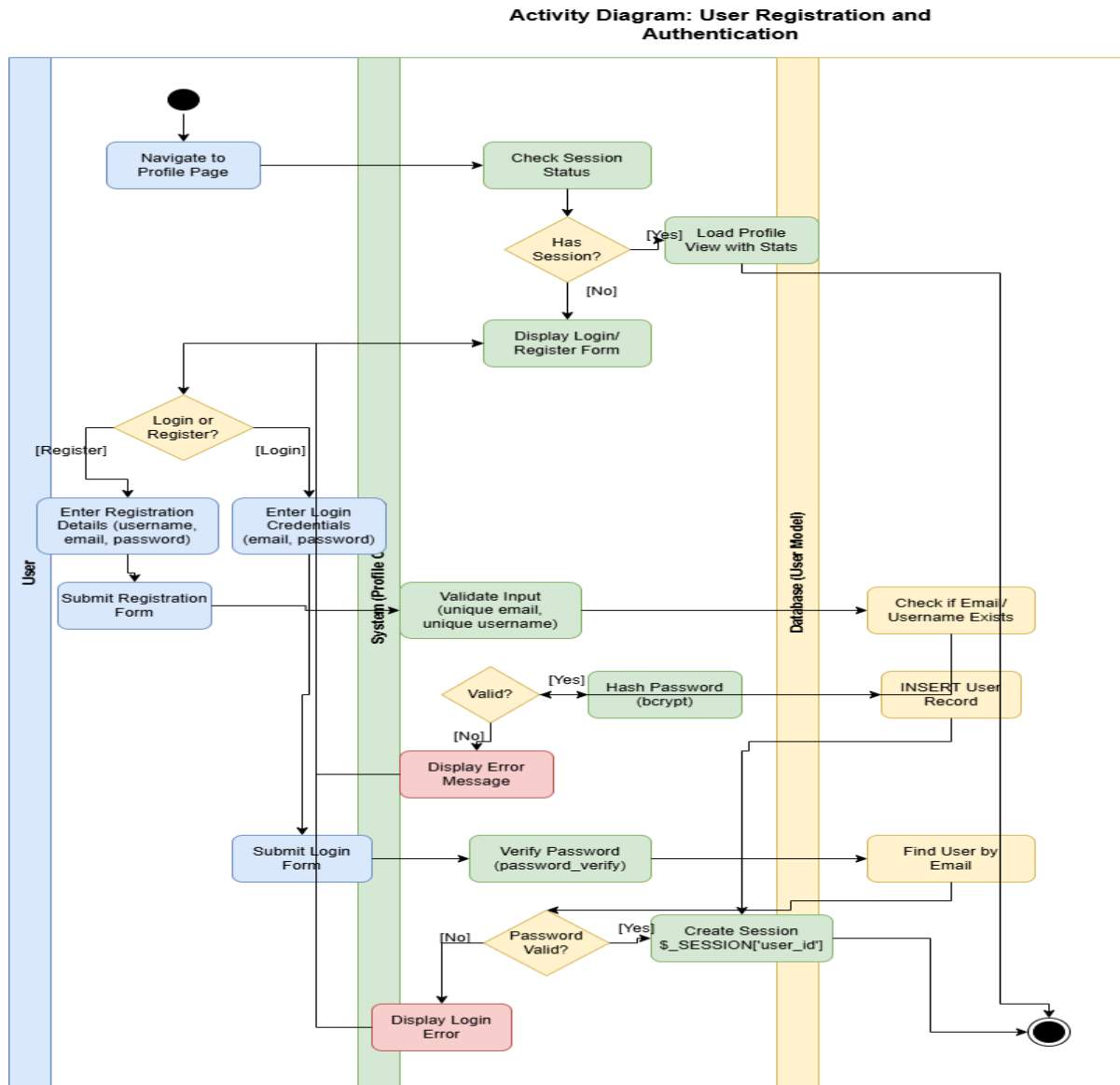
- Calculating WPM and accuracy metrics in real-time
- Managing test sessions and timers

We chose this actor hierarchy because it reflects the progressive disclosure pattern in our UI design. Guest users are encouraged to register by seeing the value proposition (leaderboard, statistics) before committing to account creation. The "extend" relationship between "Save Test Results" and "Take Typing Test" indicates that saving is an optional extension that only applies to authenticated users.

2.2 Activity Diagram

2.2.1 Activity Diagram: User Registration and Authentication

Diagram File: Activity_Authentication.drawio.xml



Documentation

This Activity Diagram illustrates the user registration and login processes.

Registration Flow:

1. User navigates to the profile page

2. System detects no active session and displays login/register form
3. User chooses to register and enters credentials (username, email, password)
4. System validates input (unique email, unique username, password requirements)
5. If validation fails, error messages are displayed
6. If validation passes, the password is hashed using bcrypt
7. User record is created in the database
8. Session is established and user is redirected to their profile

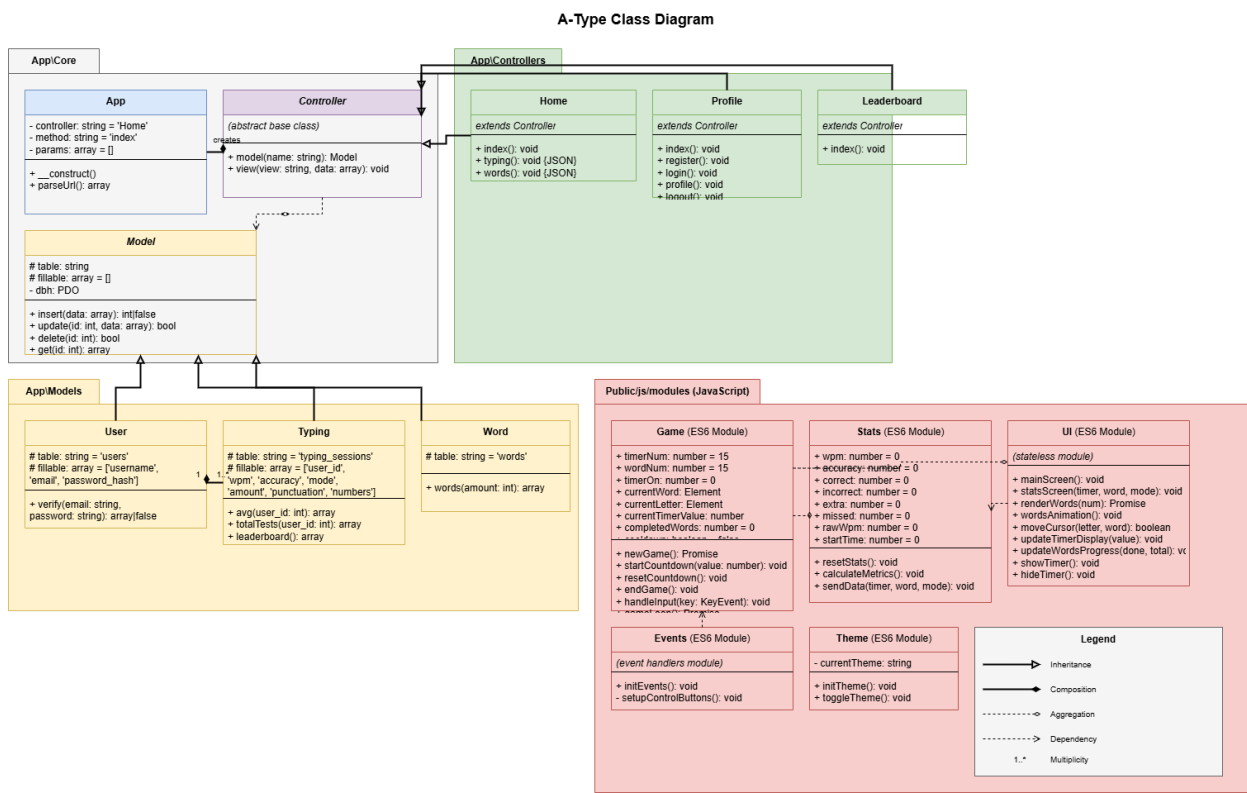
Login Flow:

1. User enters email and password
2. System retrieves user record by email
3. Password is verified using `password_verify()` against stored hash
4. If verification fails, error is displayed
5. If verification succeeds, session is established

We chose to use `password_hash()` with `PASSWORD_DEFAULT` algorithm (currently bcrypt) because it automatically handles salt generation and is forward-compatible with future PHP versions. The separation of registration and login paths with shared session establishment reflects our DRY principle in the implementation.

2.3 Class Diagram

Diagram File: `Class_Diagram.drawio.xml`



Documentation

The Class Diagram represents the complete object model of A-Type, organized according to the MVC architecture pattern.

Core Framework Classes:

- **App:** The front controller that handles routing. It parses URLs, instantiates the appropriate controller, and invokes the requested method. We chose composition with Controller because App creates and manages controller instances during request handling.
- **Controller (Abstract):** Base class providing common functionality for loading models and rendering views. Uses dependency injection pattern where models are instantiated on-demand via the `model()` method.
- **Model (Abstract):** Provides database abstraction with PDO. Implements the Active Record pattern with methods for CRUD operations. The `$fillable` array provides mass-assignment protection.

Domain Models:

- **User:** Extends Model with user-specific operations. The `verify()` method handles authentication logic. We chose to keep password verification in the model because it's domain logic that belongs with the User entity.
- **Typing (TypingSession):** Manages typing test data. Has a **composition** relationship with User because a TypingSession cannot exist without a User (enforced by foreign key with CASCADE delete). The aggregate methods (`avg()`, `totalTests()`, `leaderboard()`) encapsulate complex queries.
- **Word:** Simple model for the word bank. Has no direct relationship with other domain models as words are fetched independently for each test.

Controller Classes:

- **Home:** Handles the main typing test page, word fetching API, and result submission. The `typing()` method is an API endpoint that validates authentication before saving.
- **Profile:** Manages all authentication flows (login, register, logout) and profile display. Uses session management for state.
- **Leaderboard:** Single-responsibility controller that only fetches and displays leaderboard data.

Frontend Module Classes (JavaScript):

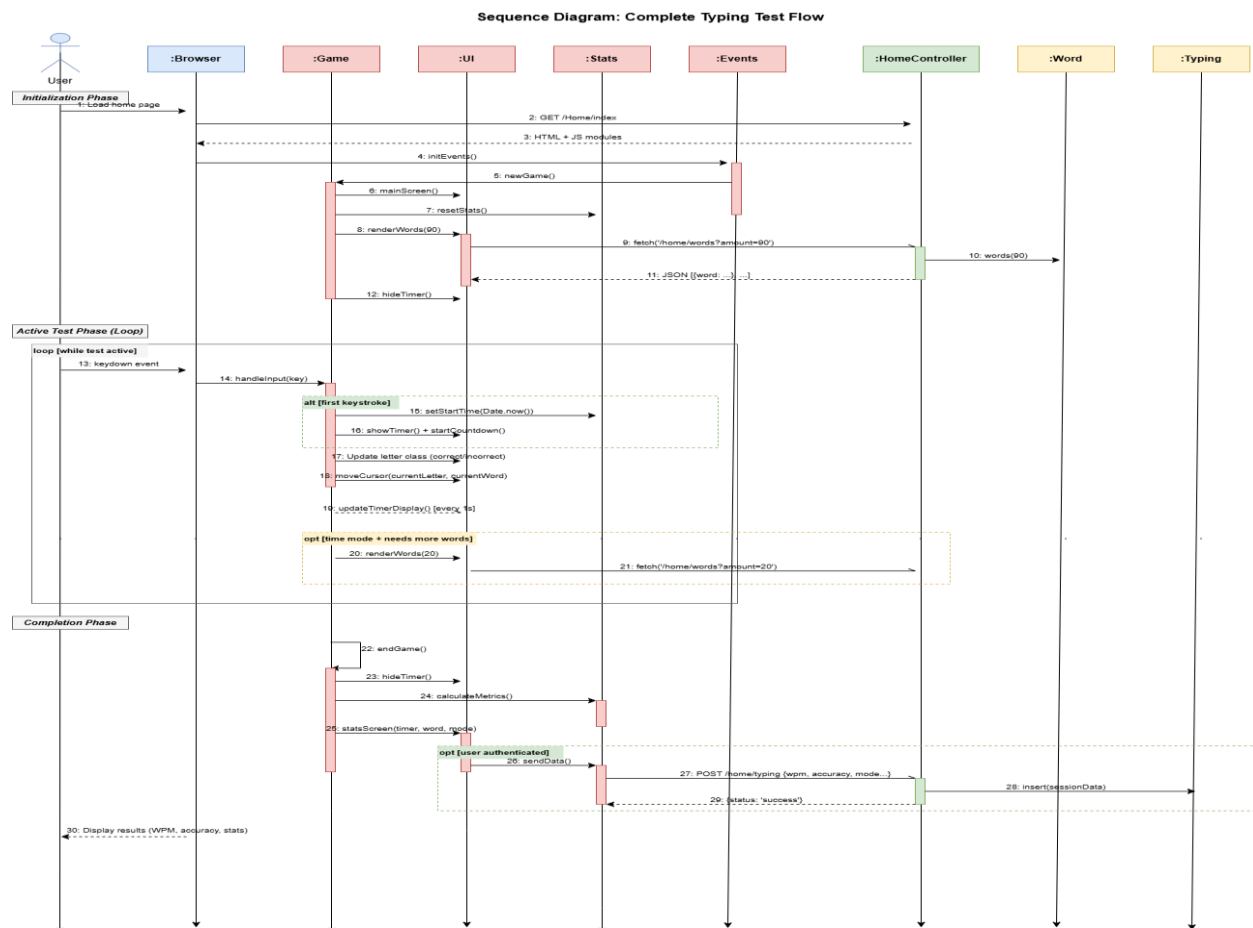
- **Game:** The state machine controller for the typing test. Manages game state variables and coordinates between UI and Stats modules. Has **aggregation** relationship with UI and Stats as it uses their functionality but doesn't own them.
- **Stats:** Calculates metrics (WPM, accuracy) and manages statistical state. Separated from Game to follow Single Responsibility Principle.
- **UI:** Handles all DOM manipulation and rendering. Depends on Stats for displaying results.
- **Events:** Event handler registration and delegation. Depends on Game for invoking game actions.

The inheritance hierarchy (Controller -> Home/Profile/Leaderboard and Model -> User/Typing/Word) follows the Template Method pattern where base classes define the skeleton of operations and subclasses provide specific implementations.

2.4 Sequence Diagrams

2.4.1 Sequence Diagram: Complete Typing Test Flow

Diagram File: Sequence_TypingTest.drawio.xml



Documentation

This Sequence Diagram traces the complete lifecycle of a typing test from page load to result submission.

Initialization Phase:

1. User requests the home page
2. Browser loads HTML and JavaScript modules
3. `initEvents()` registers all event listeners
4. `newGame()` is called to initialize the first test
5. An async fetch request retrieves random words from `/home/words`
6. Words are rendered and the cursor is positioned

Active Test Phase:

1. User presses first key, triggering `handleInput()`
2. Game module starts the countdown timer via `startCountdown()`
3. For each subsequent keystroke, the system validates the character and updates classes (correct/incorrect)
4. UI module moves the cursor and handles word scrolling
5. Timer ticks are shown via `updateTimerDisplay()`

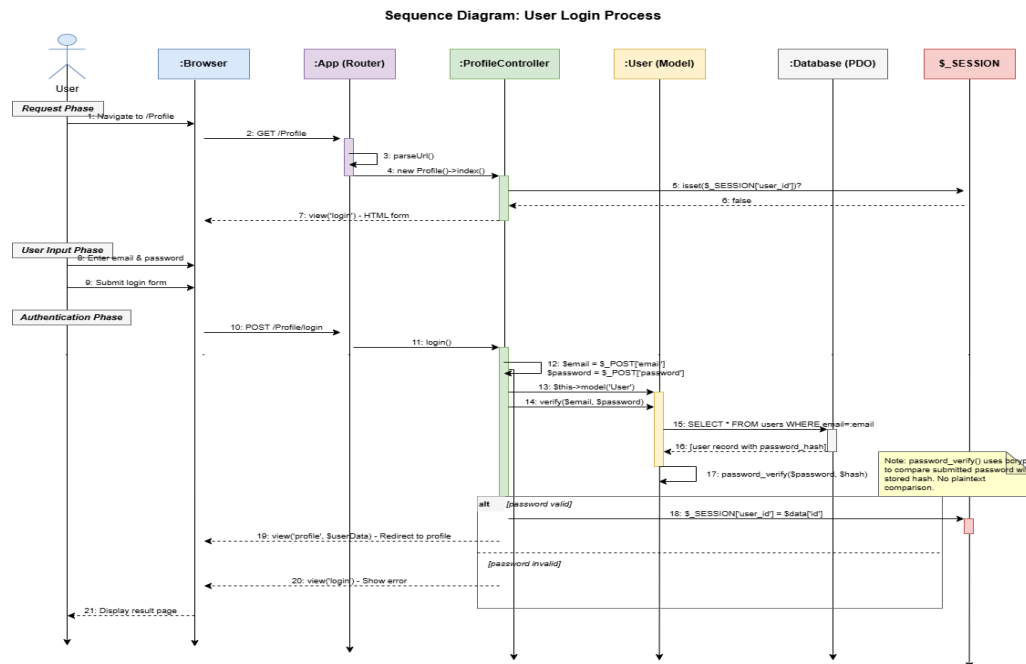
Completion Phase:

1. Timer reaches zero or all words typed, triggering `endGame()`
2. Stats module calculates all metrics via `calculateMetrics()`
3. UI switches to stats screen via `statsScreen()`
4. If user is authenticated, stats module sends POST to `/home/typing`
5. Server validates session and saves to database

We used **asynchronous messages** (half arrowheads) for the fetch calls and timer updates to accurately represent JavaScript's event-driven nature. The **opt** fragment around result saving shows the conditional behavior based on authentication status.

2.4.2 Sequence Diagram: User Login Process

Diagram File: Sequence_Login.drawio.xml



Documentation

This Sequence Diagram details the login authentication process.

Request Phase:

1. User submits login form with email and password
2. Browser sends POST to `/profile/login`
3. Apache routes request to Profile controller's `login()` method

Authentication Phase:

1. Controller instantiates User model via `$this->model('User')`
2. `verify()` method is called with credentials
3. Model executes parameterized query to find user by email
4. If user found, `password_verify()` compares submitted password with stored hash

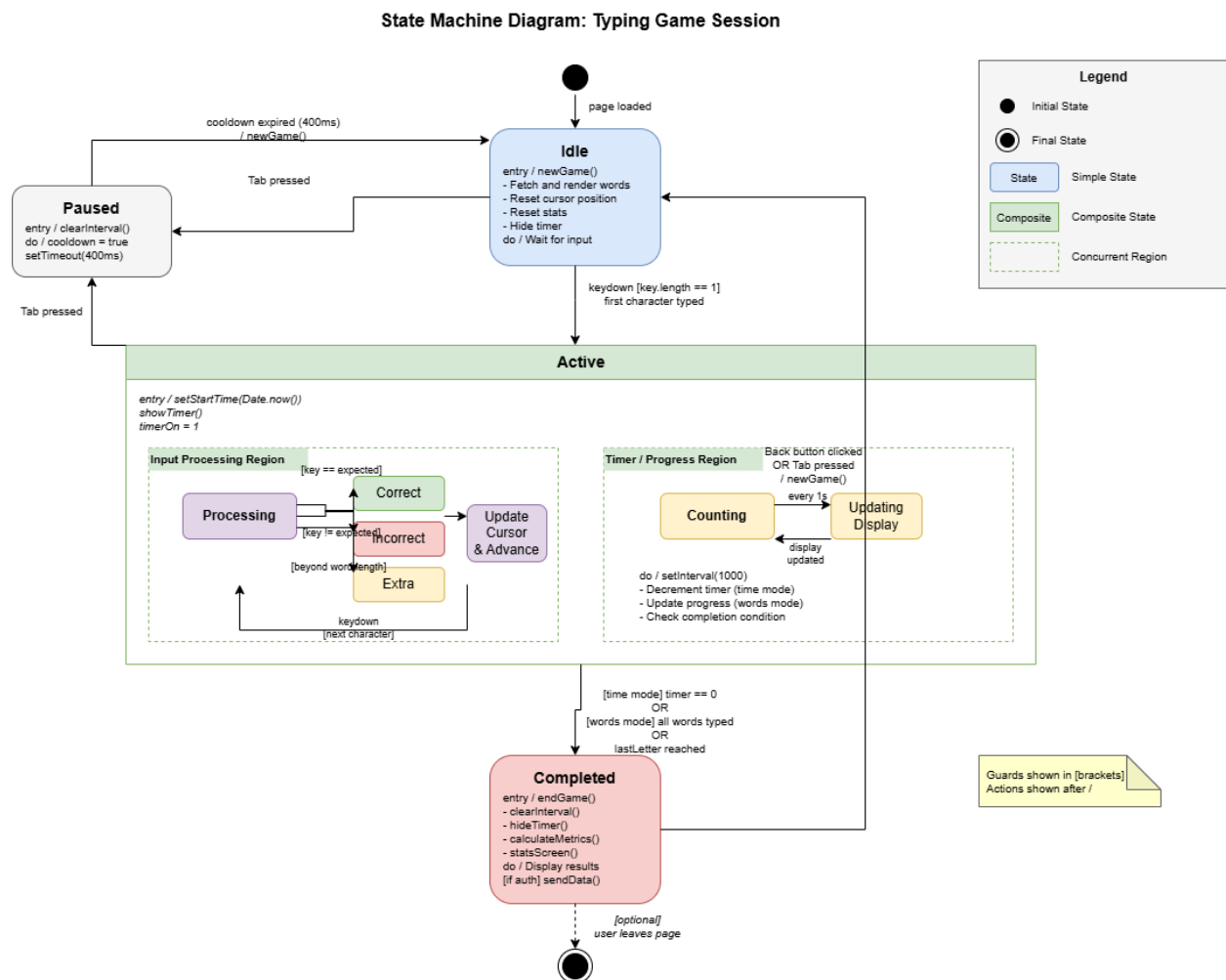
Response Phase:

1. If verification succeeds, session ID is stored via `$_SESSION['user_id']`
2. User statistics are fetched via Typing model
3. Profile view is rendered with user data
4. If verification fails, login view is re-rendered with error state

We chose to show the PDO interaction explicitly because the prepared statement pattern is critical for SQL injection prevention. The **alt** fragment clearly shows the two possible outcomes (success/failure) and their respective flows.

2.5 State Machine Diagram

Diagram File: StateMachine_TypingGame.drawio.xml



Documentation

The State Machine Diagram models the lifecycle of a typing test session, which has the most complex state behavior in the system.

States:

1. **Idle**: Initial state when page loads. The word display is populated but timer is hidden. Waiting for user input.

- Entry action: `newGame()` initializes words and cursor

- Exit action: `setStartTime()` records start timestamp
2. **Active (Composite State):** The main typing state containing two concurrent regions:
- **Input Processing Region:** Handles character validation
 - *Correct:* Letter matches expected
 - *Incorrect:* Letter doesn't match
 - *Extra:* Additional letters beyond word length
 - **Timer Region:** Manages countdown (time mode) or progress (words mode)
 - *Counting:* Timer is decrementing
 - *Updating:* Display is being refreshed
3. **Paused:** Entered when user presses Tab for quick reset. This is a transient state.
- Transition to Idle via `newGame()`
4. **Completed:** Final state when test ends.
- Entry action: `calculateMetrics()` and `statsScreen()`
 - User can transition back to Idle by clicking "Back" button

Transitions:

- Idle -> Active: First keystroke (any character key)
- Active -> Completed: Timer expires OR last word completed
- Active -> Paused: Tab key pressed
- Paused -> Idle: Cooldown timer expires (400ms debounce)
- Completed -> Idle: Back button clicked or Tab pressed

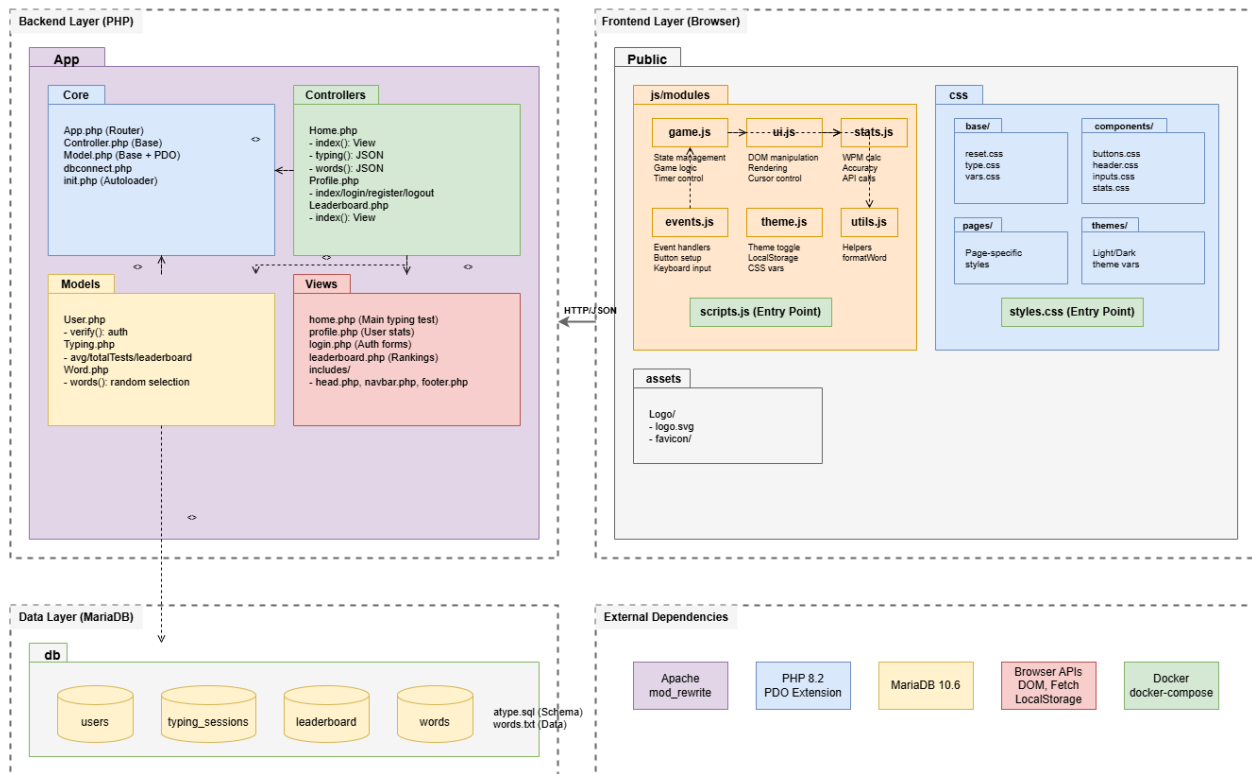
We chose to model the Active state as a composite with concurrent regions because the input processing and timer operate independently. This accurately reflects the JavaScript implementation where `setInterval` for the timer runs parallel to the `onkeydown` event handler.

The 400ms cooldown between Tab presses prevents accidental double-resets and is implemented as a guard condition on the transition.

2.6 Package Diagram

Diagram File: `Package_Diagram.drawio.xml`

Package Diagram: A-Type System Architecture



Documentation

The Package Diagram shows the logical organization of the A-Type system into subsystems.

Backend Packages (App/):

1. Core: Contains the MVC framework infrastructure

- `App.php`: Router/Front Controller
- `Controller.php`: Base controller class
- `Model.php`: Base model with database abstraction
- Dependency: Uses PDO extension for database connectivity

2. Controllers: Request handlers

- `Home.php`: Main page and API endpoints
- `Profile.php`: Authentication and user profiles
- `Leaderboard.php`: Rankings display
- Dependencies: Imports from Core (extends Controller) and Models (uses models)

3. Models: Domain entities

- `User.php`: User authentication and data
- `Typing.php`: Test session management
- `Word.php`: Word bank access

- Dependencies: Imports from Core (extends Model)

4. **Views:** Presentation layer

- Page templates (home.php, profile.php, etc.)
- Shared components in `includes/`
- Dependencies: Receives data from Controllers

Frontend Packages (`Public/`):

1. **js/modules:** JavaScript application modules

- `game.js`: Game state and logic
- `events.js`: Event handling
- `ui.js`: DOM manipulation
- `stats.js`: Metrics calculation
- `theme.js`: Theme management
- `utils.js`: Helper functions
- Dependencies: ES6 module imports between files

2. **css:** Stylesheet organization

- `base/`: Foundation styles (reset, typography, variables)
- `components/`: Reusable UI components
- `pages/`: Page-specific styles
- `themes/`: Theme variations
- Dependencies: CSS imports via `@import`

External Dependencies:

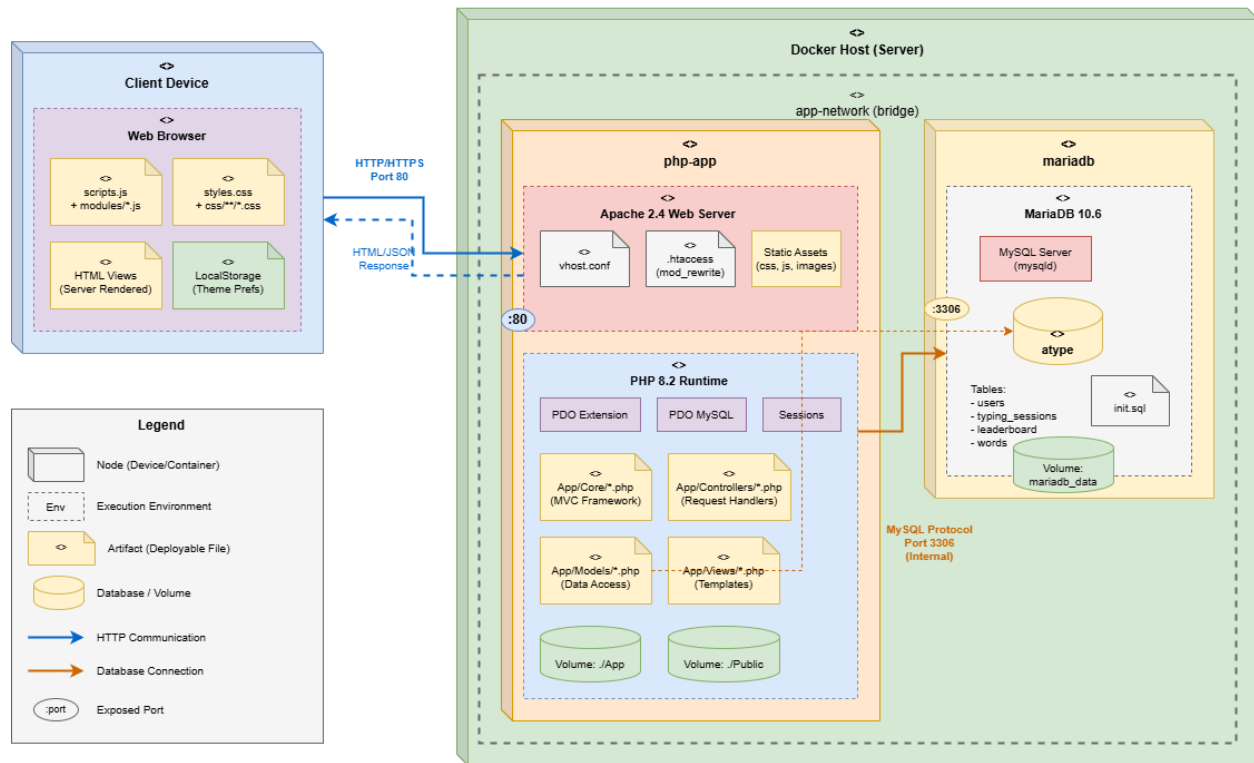
- **MariaDB:** Database server (via PDO)
- **Apache/mod_rewrite:** URL routing
- **Browser APIs:** DOM, Fetch, LocalStorage

We organized packages to follow the **Separation of Concerns** principle. Each package has a single responsibility, and dependencies flow downward (Controllers depend on Models, not vice versa). The frontend packages mirror this structure with clear module boundaries.

2.7 Deployment Diagram

Diagram File: `Deployment_Diagram.drawio.xml`

Deployment Diagram: A-Type System Infrastructure



Documentation

The Deployment Diagram illustrates the physical architecture of the A-Type system as deployed using Docker containers.

Client Node (User Device):

- **Web Browser:** Chrome, Firefox, Safari, Edge
 - Executes JavaScript modules
 - Renders HTML/CSS views
 - Stores theme preferences in LocalStorage
 - Communicates via HTTP/HTTPS

Docker Host Node:

1. php-app Container (PHP 8.2 + Apache):

- **Apache Web Server:**
 - Listens on port 80
 - mod_rewrite enabled for clean URLs
 - Serves static assets (CSS, JS, images)

- **PHP Runtime:**
 - Executes MVC application
 - PDO extension for database connectivity
 - Session management via files
- **Application Files:**
 - `/var/www/html/App/`: Backend PHP code
 - `/var/www/html/Public/`: Static assets and front controller

2. mariadb Container (MariaDB 10.6):

- **MySQL Server:** Listens on port 3306
- **Databases:** `atype` database with all tables
- **Volume:** Persistent storage via Docker volume

Communication Paths:

- **Browser <-> Apache:** HTTP on port 80
 - HTML page requests
 - API calls (JSON responses)
 - Static asset delivery
- **PHP <-> MariaDB:** MySQL protocol on port 3306
 - Persistent PDO connections
 - Parameterized queries
- **Docker Network:** Bridge network `app-network`
 - Container-to-container communication
 - DNS resolution (`mariadb` hostname)

We chose Docker containerization because:

1. **Consistency:** Development and production environments are identical
2. **Isolation:** Each service runs in its own container
3. **Scalability:** Containers can be replicated for load balancing
4. **Portability:** The entire application can be deployed anywhere Docker runs

The volume mounting of `App/` and `Public/` directories enables hot-reloading during development without rebuilding containers.

3. Architectural Justification

Chosen Architecture: 3-Tier Client/Server with MVC Pattern

A-Type implements a **3-Tier Client/Server Architecture** with the **Model-View-Controller (MVC)** pattern for the backend application layer.

Architectural Layers

PRESENTATION TIER

(Browser: HTML, CSS, JavaScript ES6 Modules)

HTTP/JSON

APPLICATION TIER

(PHP 8.2 + Custom MVC Framework)

Controllers

Models

Views

PDO/MySQL

DATA TIER

Justification

1. Separation of Concerns

The 3-tier architecture enforces clear boundaries between presentation, business logic, and data storage. This separation provides several benefits:

- **Independent Development:** Frontend developers can work on the JavaScript modules and CSS without affecting backend logic. Backend developers can modify PHP code without breaking the UI.
- **Testability:** Each tier can be tested independently. Models can be unit tested without views, controllers can be tested with mock models.
- **Maintainability:** Changes to the database schema only require updates to the Model layer, not the entire application.

2. MVC Pattern Benefits

Within the application tier, the MVC pattern provides additional structure:

Model (Active Record Pattern):

- Encapsulates all database operations
- Provides mass-assignment protection via `$fillable` array
- Uses prepared statements preventing SQL injection
- Domain logic stays with entities (e.g., password verification in User model)

View (PHP Templates):

- Simple PHP templates with data injection
- Reusable includes for headers, footers, navigation
- Clear separation from business logic

Controller (Thin Controllers):

- Orchestrate model and view interactions
- Handle HTTP request/response concerns
- Authentication checks and redirects

3. Scalability Considerations

The architecture supports horizontal scaling:

- **Stateless Application Tier:** PHP sessions can be externalized to Redis for multi-server deployments
- **Database Replication:** MariaDB supports master-slave replication for read scaling
- **Static Asset CDN:** CSS, JS, and images can be served from CDN without changing application code

- **Container Orchestration:** Docker containers can be managed by Kubernetes for auto-scaling

4. Security Architecture

The layered architecture enhances security:

- **Input Validation:** Controllers validate and sanitize input before passing to models
- **SQL Injection Prevention:** PDO prepared statements in the Model layer
- **Password Security:** bcrypt hashing with automatic salt
- **Session Management:** PHP native sessions with configurable timeout
- **XSS Prevention:** Output escaping in views

5. Why Not Microservices?

We considered a microservices architecture but chose MVC for the following reasons:

1. **Project Scope:** A-Type is a focused application with a small domain. Microservices would introduce unnecessary complexity.
2. **Development Team Size:** For a small team, maintaining multiple services, API gateways, and service discovery adds overhead without proportional benefits.
3. **Deployment Complexity:** Microservices require container orchestration (Kubernetes), service mesh, and distributed tracing. Our Docker Compose setup is simpler.
4. **Performance:** Inter-service communication adds latency. A monolithic MVC application has lower overhead for our use case.

6. Why Not SPA (Single Page Application)?

We use ES6 modules with server-rendered views rather than a full SPA framework:

1. **SEO:** Server-rendered pages are better for search engine indexing
2. **Initial Load Time:** No framework bundle to download before first render
3. **Simplicity:** Native JavaScript modules avoid build tool complexity
4. **Progressive Enhancement:** The application works without JavaScript for basic navigation

Architecture Alignment with Project Goals

Goal	How Architecture Helps
------	------------------------

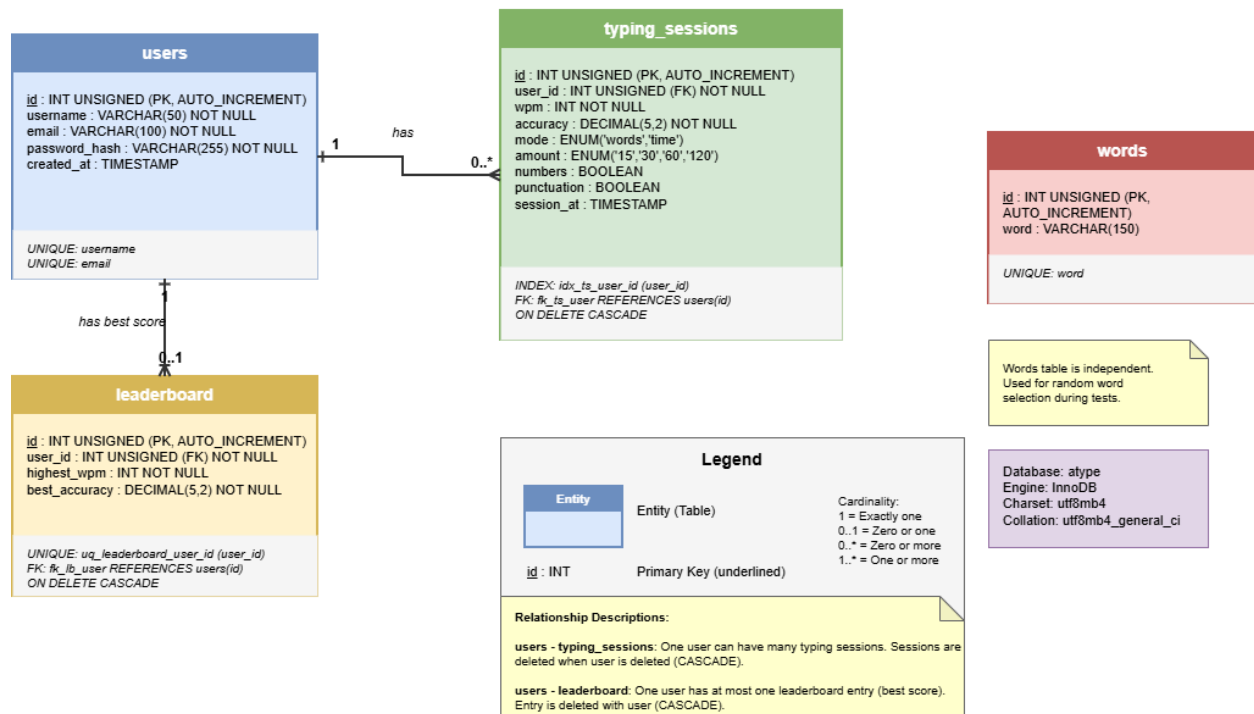
Goal	How Architecture Helps
Accurate WPM Measurement	Stats module is isolated and testable
Multiple Test Modes	Game state machine handles mode switching cleanly
User Progress Tracking	Model layer provides consistent data access patterns
Competitive Leaderboard	Aggregate queries in Typing model encapsulate ranking logic
Responsive UI	Separated CSS architecture with base/components/themes
Easy Deployment	Docker containers encapsulate entire stack

4. Database Design

4.1 Entity-Relationship Diagram

Diagram File: `ERD_Diagram.drawio.xml`

Entity-Relationship Diagram: A-Type Database



Documentation

The ERD shows four entities with their relationships:

Users (Strong Entity):

- Primary Key: `id` (auto-increment)
- Unique Constraints: `username`, `email`
- Stores authentication credentials

TypingSessions (Weak Entity):

- Primary Key: `id` (auto-increment)
- Foreign Key: `user_id` references `Users(id)`
- Stores individual test results
- Relationship: Many-to-One with `Users` (a user has many sessions, a session belongs to one user)

Leaderboard (Weak Entity):

- Primary Key: `id` (auto-increment)
- Foreign Key: `user_id` references `Users(id)` (UNIQUE)
- Stores best scores per user
- Relationship: One-to-One with `Users` (each user has at most one leaderboard entry)

Words (Strong Entity):

- Primary Key: `id` (auto-increment)
- Unique Constraint: `word`
- Independent word bank, no relationships to other entities

The **CASCADE DELETE** on foreign keys ensures referential integrity. When a user is deleted, all their typing sessions and leaderboard entries are automatically removed, preventing orphaned records.

4.2 Third Normal Form (3NF) Analysis

First Normal Form (1NF)

Requirements:

- All columns contain atomic values
- No repeating groups
- Primary key defined

Analysis:

Table	1NF Compliance	Notes
users	✓ Yes	All columns atomic (username, email, password_hash are single values)
typing_sessions	✓ Yes	All columns atomic. <code>mode</code> is ENUM (single value), not a list
leaderboard	✓ Yes	All columns atomic
words	✓ Yes	Single word per row

Second Normal Form (2NF)

Requirements:

- Must be in 1NF
- No partial dependencies (non-key columns must depend on the entire primary key)

Analysis:

All tables use single-column primary keys (`id`), so partial dependencies are impossible by definition. If we had composite keys, we would need to verify that all non-key attributes depend on the entire key.

Table	2NF Compliance	Notes
users	✓ Yes	Single PK, all attributes depend on <code>id</code>
typing_sessions	✓ Yes	Single PK, all attributes depend on <code>id</code>
leaderboard	✓ Yes	Single PK, all attributes depend on <code>id</code>
words	✓ Yes	Single PK, <code>word</code> depends on <code>id</code>

Third Normal Form (3NF)

Requirements:

- Must be in 2NF
- No transitive dependencies (non-key columns cannot depend on other non-key columns)

Analysis:

users table:

- `username` depends only on `id`, not on `email`
- `email` depends only on `id`, not on `username`

- `password_hash` depends only on `id`
-  **No transitive dependencies**

typing_sessions table:

- `wpm`, `accuracy`, `mode`, `amount`, `punctuation`, `numbers` all depend only on the `session id`
- None of these attributes depend on each other (e.g., `wpm` doesn't determine `accuracy`)
- `user_id` is a foreign key, not a transitive dependency
-  **No transitive dependencies**

leaderboard table:

- `highest_wpm` and `best_accuracy` depend on `id` (representing the user's best performance)
- These could potentially be computed from `typing_sessions`, but storing them separately is a deliberate denormalization for query performance
-  **Acceptable - Controlled denormalization for performance**

words table:

- Single non-key attribute (`word`) depends only on `id`
-  **No transitive dependencies**

Normalization Summary

Table	1NF	2NF	3NF
users	✓	✓	✓
typing_sessions	✓	✓	✓
leaderboard	✓	✓	✓*
words	✓	✓	✓

* Controlled denormalization for performance

SQL Schema

```
-- MariaDB-safe schema (avoids reserved words, adds FKs/indexes, re-runnable)
```

```
CREATE DATABASE IF NOT EXISTS `atype`  
  CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;  
USE `atype`;
```

```
-- Users
```

```
CREATE TABLE IF NOT EXISTS `users` (  
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `username` VARCHAR(50) NOT NULL,  
  `email` VARCHAR(100) NOT NULL,  
  `password_hash` VARCHAR(255) NOT NULL,  
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uq_users_username` (`username`),  
  UNIQUE KEY `uq_users_email` (`email`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5. Implementation Overview

Technology Stack

Layer	Technology	Version
Frontend	HTML5, CSS3, JavaScript (ES6)	-
Backend	PHP	8.2
Database	MariaDB	10.6
Web Server	Apache	2.4
Containerization	Docker + Docker Compose	-

Code Structure Alignment with Package Diagram

A-type/

```
|— App/                                # Backend Package
|   |— Core/                          # Core Framework Package
|   |   |— App.php                    # Router
|   |   |— Controller.php            # Base Controller
|   |   |— Model.php                 # Base Model (Active Record)
|   |— Controllers/                  # Controllers Package
|   |   |— Home.php                  # Main + API
|   |   |— Profile.php               # Auth + Profile
|   |   |— Leaderboard.php           # Rankings
|   |— Models/                      # Models Package
|   |   |— User.php                  # User Entity
|   |   |— Typing.php                # Session Entity
|   |   |— Word.php                  # Word Bank
|   |— Views/                        # Views Package
|       |— home.php
|       |— profile.php
|       |— login.php
|       |— leaderboard.php
```

Key Implementation Features

1. **Routing:** URL pattern `/controller/method/params` parsed in `App.php`
2. **Database:** PDO with prepared statements, persistent connections
3. **Authentication:** bcrypt password hashing, PHP sessions
4. **API Endpoints:** JSON responses for word fetching and result submission
5. **Real-time Metrics:** Client-side WPM/accuracy calculation in `stats.js`
6. **Game State:** State machine pattern in `game.js` with exported state variables
7. **Responsive Design:** CSS custom properties and `clamp()` for fluid scaling

This report was prepared as part of the software engineering course project documentation.